École nationale supérieure de l'électronique et de ses applications (ENSEA)

FINAL YEAR THESIS FOR ENGINEERING DEGREE

THEME

Large Language Model in healt

Directed By :

- FUZELLIER Thomas

Supercised By :

- JEBRI Ayoub

Academic Year 2023 – 2024

UNIVERSIDAD POLITÉCNICA DE MADRID

Table of contents

Acknowledgments

First and foremost, I would like to express my profound gratitude to everyone who contributed to making my internship at MEDAL, located within the Biomedical Technology Center in Madrid, an exciting, enriching, and instructive experience. Thanks to their support and expertise, this professional journey unfolded under the most optimal conditions.

I would like to extend my heartfelt thanks to Mrs. Ernestina MENASALVAS, Professor and Ph.D. at the Polytechnic University of Madrid. Her guidance throughout my internship was invaluable. She enabled me to deepen the knowledge I had acquired during my studies while introducing me to new, equally fascinating areas of computer science. Her extensive experience as a researcher and educator has been a source of inspiration and provided me with a valuable insight into the professional world in disciplines I am passionate about, such as data processing and artificial intelligence.

My thanks also go to Mrs. Cristina HEATH, who manages the administrative and secretarial functions of the MEDAL group. Her invaluable assistance, particularly in administrative matters, made my integration into the lab team and my settlement in Spain much smoother.

I would like to express my gratitude to Mr. Ayoub JEBRI, my academic supervisor at ENSEA. His insightful advice and support were instrumental in the writing of this report. Despite the distance, his availability and kindness were essential supports throughout this journey.

I must also mention Mrs. Maria-Esther VIDAL, one of my supervisors during my stay in Hanover, Germany, where I participated in training on knowledge graphs. Her supervision significantly contributed to my academic and professional development.

A special thanks to Mr. Enrique SOLERA NAVARRO and Mr. Alberto GONZALEZ CALATAYUD, Ph.D. candidates within the MEDAL group. Their expertise in the field of large language models, along with their shared experiences and our discussions, served as a valuable source of motivation throughout this experience.

I am also deeply grateful to the entire laboratory team for their warm welcome and collaborative spirit. The regular interactions with other project teams allowed me to enrich my ideas, identify areas for improvement, and integrate new perspectives. Their generosity, including inviting me to their meetings and events, gave me a comprehensive and enriching understanding of their work.

Finally, I would like to thank Mr. Eneko CHIPI, who introduced me to Mrs. MENASALVAS. Without his invaluable help, this internship would not have been possible. His support greatly facilitated my move to Madrid.

In conclusion, I am deeply appreciative of all those who contributed, directly or indirectly, to this journey at MEDAL. Their guidance and support were crucial in my personal and professional growth. This internship was a formative and inspiring experience, and I am confident it will serve as a significant stepping stone in my engineering career.

Introduction

Personal Presentation and Internship Objectives

My name is Thomas FUZELLIER, and I am a final-year student at ENSEA (École Nationale Supérieure de l'Électronique et de ses Applications), specializing in Computer and System Engineering. This end-of-studies internship is a crucial step in my academic journey, allowing me to apply my skills while confronting real-world issues in a professional setting.

My internship, titled "LLM in Health," is situated in a rapidly expanding field: the use of advanced language models (LLM) in the medical sector. The goal is to develop an innovative solution to assist clinicians in their daily work. Specifically, I am designing a tool that facilitates the analysis of clinical notes related to septicemia cases, aiming to automate the extraction and interpretation of essential information to improve decision-making and optimize medical processes.

During this internship, I had the opportunity to contribute directly to a high-impact topic, combining technical, medical, and human challenges. This experience has enhanced my project management skills, communication in an international context, and adaptation to innovative professional environments.

General Context of the MEDAL Laboratory and the Internship Subject

The MEDAL Laboratory (Machine Learning and Data Analysis Laboratory), located in Madrid, is renowned for its expertise in artificial intelligence applied to medical sciences. Led by researchers and experts in machine learning, it collaborates closely with medical institutions to meet the challenges of modern medicine.

Sepsis, which is at the heart of this project, is a complex medical emergency that requires rapid and accurate management. Due to its multifactorial nature and various clinical presentations, the diagnosis and monitoring of patients often rely on a careful analysis of available data, including clinical notes. These documents, while rich in information, require expert reading and can be subject to variable interpretations by practitioners.

The approach we adopt at MEDAL aims to leverage the potential of advanced language models to simplify and enhance this analysis. Our work is part of an innovation perspective, where artificial intelligence technologies do not replace clinicians but act as decision-support tools. This project represents a unique opportunity to intersect the fields of fundamental research, technological development, and societal impact, with the ultimate ambition of improving the care provided to patients.

Image 1 **: C**entro de Tecnología Biomédica (CTB)

Internship Context

Importance of LLMs in the Medical Field

Large Language Models (LLMs) are transforming how healthcare professionals interact with medical data. Models such as GPT and Llama are capable of analyzing massive volumes of unstructured textual data, including clinical notes, to extract actionable insights. In critical cases like sepsis, where rapid response is essential, these models automate complex tasks such as classification, named entity recognition (NER), and clinical information synthesis, thereby reducing the time required for informed medical decisions.

Moreover, LLMs facilitate more personalized medicine by adapting to the specific context of each patient, highlighting subtle patterns, and interpreting results in complex clinical scenarios. Their ability to process and interpret heterogeneous data is particularly valuable in hospital environments, where healthcare professionals often face limited resources and scattered information.

Artificial intelligence, especially in the medical domain, is experiencing rapid growth, playing an increasingly pivotal role in healthcare. As shown in the graph below, the adoption of AI solutions in health has risen significantly in recent years. This trend is driven by technological advancements, the increased availability of digital medical data, and the need to optimize patient care while reducing costs. LLMs fit seamlessly into this dynamic, exemplifying how AI can address current challenges in the medical sector and improve the quality of care.

It is precisely within this expanding field that my final-year internship took place. Conducted in Madrid under the supervision of Mrs. Ernestina Menasalvas, the project focused on applying Large Language Models in healthcare. This work explored the potential of these technologies to analyze complex medical data and contribute to tangible advancements in this critical sector.

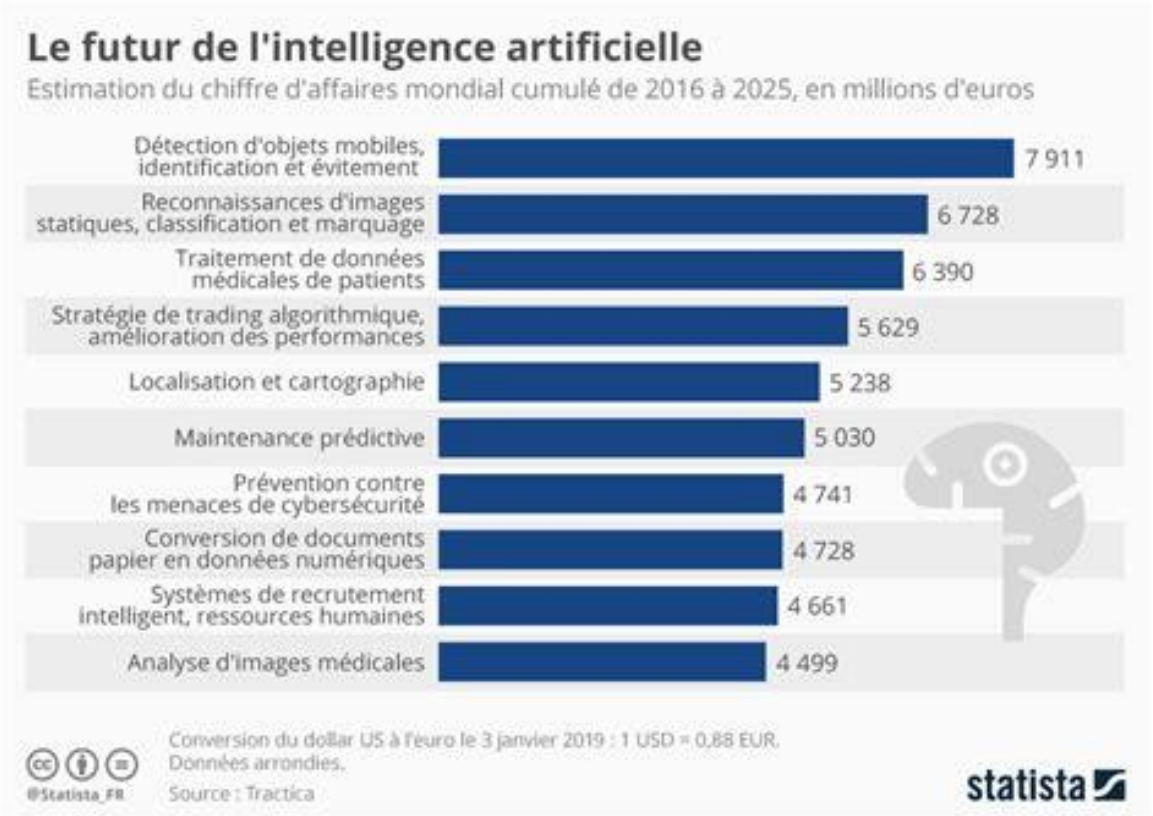

Image 2 : Importance of AI in Different Fields

Problem Statement: Analysis of Sepsis Clinical Notes

Sepsis is a medical emergency where the success of treatment relies heavily on the speed and accuracy of diagnosis. However, clinical notes, often written without standardization, hold a wealth of information that remains underutilized due to their complexity and unstructured format.

The primary challenge addressed in this project is transforming these textual data into actionable insights to assist clinicians in answering critical questions such as:

- What is the source of the infection?
- What are the results of the cultures?
- Which antibiotic treatments have been administered?

Using configurable prompts to interact with advanced models such as Llama2 and Llama3, the goal is to reliably extract these key pieces of information while providing justifications based on the available data. This approach aims to enhance clinical decision-making and optimize the management of sepsis cases.

Overview of Tools and Technologies Used

Adopted Technologies

1. **Language Models (LLMs):**
    - **Llama2 and Llama3:** Chosen for their robustness and ability to handle domain-specific medical data, these models excel in Named Entity Recognition (NER) tasks and contextual analysis.
    - **Tiktoken and Hugging Face Transformers:** Utilized for token management, ensuring compliance with the input limits of the LLMs.
2. **Python and Its Libraries:**
    - **pandas:** For structured data manipulation and format conversion.
    - **requests:** To manage API calls for querying the models.
    - **openpyxl:** To save results in Excel format.
3. **User Interface (PyQt5):**
    - Simplifies user interaction, allowing the selection of clinical files, application of prompts, and visualization of results.
4. **Custom-Developed Modules:**
    - **file_reader.py:** Handles various file formats (text, JSON, Excel, CSV) and converts them into standardized strings.
    - **process.py:** Manages file processing, prompt handling, and result extraction.
    - **save_manager.py:** Centralizes the logic for saving data in Excel or JSON formats.

Alternative Technologies Not Used and Justifications

In this project, certain technologies and approaches, while relevant, were not utilized for specific reasons. Below are examples and the rationale for these decisions:

1. LangChain

- **Why it wasn't used:** Although LangChain is a powerful library for structuring complex interactions with LLMs, it introduces an additional abstraction layer. This could have posed challenges for fine-tuning prompts, a critical aspect of the project.
- **Why this choice:** Directly working with Llama APIs allowed for granular control over data processing and token management.

2. SpaCy and BioBERT

- **Why they weren't used:** SpaCy is excellent for NER tasks, and BioBERT specializes in biomedical text processing. However, their integration would have required a separate pipeline for non-LLM-based tasks, complicating the workflow.
- **Why this choice:** Llama2/3 provided a unified approach for both entity extraction and contextual reasoning, avoiding tool fragmentation.

3. Medical Knowledge Bases (UMLS, SNOMED CT)

- **Why they weren't used:** While these resources are rich in medical semantics, integrating them would have required an additional step to map LLM-generated results to standardized terms.
- **Why this choice:** The project focused on leveraging raw textual data from clinical notes, where such knowledge bases were less critical compared to scenarios requiring advanced normalization.

4. Google Cloud Healthcare API

- **Why it wasn't used:** This API offers advanced functionalities for healthcare data processing, but relying on it would have introduced cloud dependency, incompatible with Llama2's local configuration.
- **Why this choice:** A standalone solution ensured data privacy and avoided limitations tied to cloud-based services.

5. Advanced GUI Frameworks (Dash, Streamlit)

- **Why they weren't used:** These frameworks are suitable for interactive dashboards but would have shifted the project's focus toward visualization rather than clinical analysis.
- **Why this choice:** PyQt5 provided a lightweight interface sufficient for user needs without diverting attention from core analytical objectives.

6. GPT-4 (and other proprietary models like Bard, Claude, etc.)

- **Why it wasn't used:**
    - **Cost:** GPT-4 incurs significant API costs, especially for projects involving intensive testing or large-scale processing.
    - **Cloud dependency:** GPT-4 requires a constant connection to OpenAI's API, posing security and privacy concerns, especially in medical contexts.
    - **Limited control:** As a closed model, GPT-4 does not support direct customization or fine-tuning, unlike open-source alternatives like Llama2/3.

- **Why Llama was chosen:**
    - **Open source:** Llama2/3 (by Meta) offers full transparency, enabling adaptation to specific needs, such as medical contexts.
    - **Local deployment:** Running the models locally ensured greater security and confidentiality for sensitive clinical notes.
    - **Resource efficiency:** Llama is optimized for environments with limited resources while maintaining competitive performance.
    - **Prompt flexibility:** Llama supports precise results through well-crafted prompts, critical for extracting clinical information tailored to specific requirements.
    - **Cost-effectiveness:** Once set up locally, Llama incurs no additional costs, ideal for academic projects or internships.

In summary, the choice of Llama over GPT or other proprietary models was driven by the need for control, cost-effectiveness, confidentiality, and flexibility. Llama provides a powerful and adaptable open-source alternative, aligned with the project's objectives and constraints.

Conclusion on Technology Choices

The selection of technologies balanced simplicity, efficiency, and project objectives within a limited timeframe. Tools like PyQt5, pandas, and Llama3 enabled the development of a robust pipeline, offering flexibility for prompt adjustments and the exploration of diverse clinical cases. While alternative tools could have added complementary features, their integration would have introduced unnecessary complexity or conflicted with local infrastructure constraints.

The chosen stack proved well-suited for achieving the project's goals, emphasizing adaptability, data privacy, and cost-effectiveness.

Corporate Sociale Responsibility (CSR)

Impact of the Project on Public Health

The project developed at the MEDAL laboratory leverages artificial intelligence technologies, particularly large language models (LLMs), for the analysis of clinical notes related to sepsis. This work holds significant public health implications, given the urgency and severity of sepsis, which can result in high mortality rates if not treated promptly. By automating the extraction of critical information and accurately identifying sepsis cases, our project aims to reduce the time required for an accurate diagnosis, enabling faster and more targeted medical intervention.

Integrating this technology into hospital workflows could potentially alleviate the workload of medical teams, allowing them to focus on the most critical aspects of patient care while reducing possible human errors caused by the overwhelming volume of data that must be manually processed. By streamlining and accelerating the diagnostic process, the project directly contributes to improving patient survival rates and optimizing medical resources, which is essential in a context where healthcare systems are often under strain.

Reflections on the Ethics of Using AI in Medicine

The implementation of artificial intelligence in medical practice raises significant ethical challenges. During this internship, a thorough reflection on these issues was undertaken, particularly concerning data confidentiality. Respecting patient privacy is paramount, and all processed data were rigorously anonymized prior to use. Strict security measures were implemented to prevent data breaches or unauthorized access.

Another critical ethical consideration is informed consent. Patients must be fully aware of how their data is being used, including the fact that AI technologies may analyze their medical information. This consent must be obtained transparently and in a manner that is easy to understand.

Finally, there is the issue of reliance on automated decisions. While LLMs can provide recommendations based on large volumes of data, it is vital that these tools are used as assistants rather than replacements for human clinical judgment. Healthcare professionals must remain the ultimate decision-makers, using AI as a support tool for their assessments rather than as the final authority.

These ethical reflections are essential to ensure that the deployment of AI in medicine is carried out responsibly, always prioritizing the interests and safety of patients. These concerns were regularly discussed during team meetings, incorporating feedback from medical practitioners and ethics specialists, to ensure that our approach remained aligned with the highest ethical standards.

Astonishment Report

Observations on the Functioning of the Laboratory

During my integration into the MEDAL laboratory, I was particularly impressed by the collaborative organization of the teams. The lab members emphasize key principles such as scientific rigor and technological innovation. Each project benefits from a synergy between researchers and engineers, creating a highly motivating dynamic. However, I also noted that decision-making cycles, particularly regarding the adoption of new technologies, can be quite lengthy. This is likely due to the need for rigorous scientific validation, but it could be improved to respond more rapidly to project needs.

Another observation is the strong international focus, with close collaborations with other institutions, such as during my mission in Hanover. This openness allowed me to discover new approaches, particularly in knowledge graphs, and apply them to the lab's projects.

On the other hand, access to technical resources (such as powerful servers or specialized software tools) is sometimes limited. This required careful organization and task planning adaptations. Despite these constraints, the working environment is stimulating and fosters skill development.

Discovery of Technologies and Challenges Encountered

My internship allowed me to work with cutting-edge technologies in the field of large language models (LLMs) and artificial intelligence applied to medicine. Interactions with models such as Llama2 and Llama3 gave me an in-depth understanding of their functioning, potential, and limitations. Specifically, managing prompts and analyzing results through advanced tokenization approaches were enriching yet complex tasks.

Among the challenges encountered, I identified the need to adapt prompts to meet specific requirements, such as extracting clinical features or justifying responses. This work required constant interaction between understanding medical needs and addressing the limitations of the models. For instance, analyzing clinical notes had to be optimized to comply with token constraints, leading me to test and compare various tokenizers such as AutoTokenizer, LlamaTokenizer, and Tiktoken.

Another significant aspect was the development of a user-friendly interface using PyQt5 to simplify the analysis of clinical notes. While this was a major step forward in making the tool accessible, I faced technical challenges related to integrating various features, such as managing output formats (Excel and JSON).

Finally, the unit tests I developed for the various modules ensured the reliability and maintainability of the code. However, covering all possible scenarios required considerable time, particularly for handling errors in API calls and processing files in different formats. These steps taught me to be rigorous in development and to anticipate potential issues effectively.

Internship Mission

Integration Phase

Discovering Madrid, the Laboratory, and the Teams

My final internship began with an integration phase at the MEDAL laboratory, located within the Biomedical Technology Center of the Polytechnic University of Madrid (UPM). Upon arriving in Madrid, I was welcomed by a dynamic and culturally rich city. This immersion allowed me to discover a new living and working environment.

Within the laboratory, the atmosphere was friendly and stimulating, combining innovation and applied research. I was introduced to my supervisors, Ms. Ernestina Menasalvas and Alberto Gonzalez Calatayud, as well as to the multidisciplinary team working on various projects related to artificial intelligence and bioinformatics. I also met other interns and researchers, which facilitated my integration.

Initial Training on LLMs and APIs

From the first few weeks, my integration into the MEDAL laboratory was marked by a structured training plan, developed by the team to help me master the tools necessary for my project. The main objective was to gain an in-depth understanding of large language models (LLMs), their interaction with APIs, and more broadly, the fundamental concepts of artificial intelligence (AI).

The team provided me with a list of resources and online training platforms tailored to these needs, including:

- **Coursera and edX**, for introductory and advanced courses on artificial intelligence and machine learning.
- **Hugging Face**, to discover frameworks and libraries dedicated to LLMs.
- Official documentation on models such as **Llama2 and Llama3** to understand their architecture, capabilities, and constraints.

Progressive Approach

This training took place in several stages to solidify my knowledge:

1. **Introduction to Fundamental Concepts**: I started with theoretical courses to understand the basics of language models, including how they work (transformers, attention mechanisms), how they are trained, and how they are adapted to specific tasks such as classification or text generation.
2. **Working with APIs**: Once the basics were established, I learned to interact with REST APIs, which connect language models to applications. The training included :
    - Basics of HTTP requests (GET, POST).
    - Sending structured prompts and handling JSON responses.
    - Best practices for minimizing latency and maximizing API call efficiency.
3. **Token Management**: One of the main challenges of LLMs is managing tokens, which represent fragments of text analyzed by the model. I learned how to:

- Calculate the number of tokens generated by a text using libraries like Transformers.
- Optimize the size of prompts to stay within the limits imposed by the models, while obtaining complete responses.

Practical Applications

Through these trainings, I was able to quickly put the concepts I had learned into practice by working on exercises proposed by the team:

- Writing custom prompts for specific tasks, such as extracting information from clinical notes.
- Simulating local API calls to test different parameters, such as temperature (randomness control) and the maximum number of tokens for responses.

Discovering Challenges Specific to LLMs

These training sessions also allowed me to identify challenges specific to using LLMs in a medical context:

- **Bias in Models**: LLM responses may reflect biases present in their training data.
- **Limits of Contextual Understanding**: Although effective, these models require well-designed prompts to correctly interpret complex contexts.
- **Compatibility with Varied Datasets**: Clinical notes present heterogeneous formats (abbreviations, specific medical terminologies) that can complicate their analysis.

Supervision and Interaction with the Team

Throughout this phase, I benefited from supportive supervision and regular guidance from my supervisors and colleagues. They answered my questions, corrected my initial attempts, and shared their own experiences to help me progress more quickly. These interactions allowed me to thrive in a collaborative and stimulating learning environment.

This initial training not only gave me a solid foundation in LLMs and APIs, but also boosted my confidence in implementing advanced technologies to solve real-world issues, such as analyzing sepsis cases.

Creation of the Database

Why was a textual database necessary?

As part of my project, the analysis of clinical cases of sepsis relied on the use of advanced language models (LLMs) to extract specific information. The clinical notes available were mostly provided in the form of unstructured PDF files, containing textual data encapsulated in a format that was difficult for the models to directly access.
Creating a textual database was a crucial step for several reasons:

1. **Data Accessibility**:
   Natural language processing models and analysis algorithms require raw text to function efficiently. The information encapsulated in the PDF files needed to be extracted and normalized.

2. **Standardization of Formats**:
   The PDF files came from different sources and showed significant variations in their structure (e.g., page formatting, delimitation of clinical cases, language used). Converting these files into plain text helped standardize the data, ensuring its compatibility with the processing models.

3. **Facilitating Specific Queries**:
   A well-structured database allowed the preparation of data subsets for specific tasks, such as analyzing sepsis foci or culture results.

Why were several extraction programs developed?

The significant variations in the content and structure of the PDFs led me to develop different scripts, each tailored to a particular use case. These programs provided flexibility for effectively processing documents with varied characteristics:

1. **Full Extraction: pdf_to_txt.py**
   - **Objective**: Collect all content from a PDF file into a single plain text file.
   - **Use Case**: This script was used for PDFs containing relatively short clinical notes or simple reports where no segmentation was necessary.
   - **How it works**: By combining the pages of the PDF into a single text string, this script produced files that were easy to analyze with language models.

2. **Page-by-Page Extraction: pdf_to_txt_onepage.py**
   - **Objective**: Extract the content of each PDF page and save it into a separate text file.
   - **Use Case**: This program was used for large documents containing multiple cases or information structured by pages. It allowed for fine granularity, useful for analyzing specific pages without processing the entire document.
   - **How it works**: Each page of the PDF was extracted independently and saved as a text file, simplifying workflows that required targeted analysis of specific pages.

3. **Segmentation by Clinical Case: pdf_to_txt_CASOCOMPLETO.py**
   - **Objective**: Split a PDF file into segments corresponding to distinct clinical cases, using a specific keyword ("CASO COMPLETO").

- **Use Case**: Designed for complex clinical documents containing multiple cases in a single file. Each segment represented a unique case, allowing independent and more focused analysis.
- **How it works**: After extracting all the text from the PDF, the script scans the content for the keyword "CASO COMPLETO" to segment the text. Each segment is saved as a separate text file, ensuring clear organization of the data.

Results Obtained and Advantages of the Scripts

Thanks to these scripts, I was able to transform a collection of unstructured PDF files into a rich textual database ready for exploitation. The benefits obtained included:

- **Time Savings**: The modularity of the scripts allowed for the automation of the conversion of hundreds of pages quickly.
- **Flexibility**: Depending on the project's needs, I could choose the most appropriate extraction mode (global, page-by-page, or by segment).
- **Increased Accessibility**: The generated text files were directly usable by language models, making subsequent analysis stages easier.

In summary, the creation of this textual database was a key step in the project, laying the foundation for subsequent phases, including the application of language models and the optimization of prompts.

Exploration of LLMs

Context and Objectives

Exploring language models (LLMs) was a key step in my internship. Before integrating these tools into an application dedicated to analyzing clinical notes related to sepsis, it was essential to understand how they work, their capabilities, and their limitations. This phase allowed me to select an optimal model while acquiring a deep understanding of LLM principles.
The main goal was to identify a model capable of extracting relevant information from complex data while providing clear justifications. However, before diving into domain-specific tests, I opted for a progressive approach, starting with simple experiments to better understand these technologies.

Phase 1: Discovery and Initial Experiments

Tests with Simple Queries

To familiarize myself with LLMs, I conducted initial tests by asking simple, non-specialized questions. These tests aimed to observe how the models reacted to basic queries and evaluate their ability to provide clear and coherent answers. Here are some examples of questions I used:

- "Why is the sky blue?"
- "What is artificial intelligence?"
- "Describe the main characteristics of Mars."

These first experiments allowed me to understand:

- **Answer structure**: The models provided organized answers, often in the form of paragraphs or lists.
- **Influence of parameters**: By adjusting parameters such as temperature or frequency penalty, I observed variations in the style of responses (precise vs. creative).
- **Model limitations**: Some models gave generic or underdeveloped answers, highlighting the need for properly structured prompts.

Creation of Simple Chatbots

In the next step, I developed basic chatbots connected to LLM APIs such as Llama2, GPT-3, and GPT-4. These chatbots helped me understand:

- **Interaction management**: How the models maintain context over exchanges.
- **Importance of initial prompts**: For example, a prompt like "You are a specialized medical assistant" immediately directed the responses towards appropriate terminology.
- **Challenges related to tokens**: During prolonged exchanges, the remaining number of tokens for responses became critical.

Phase 2: In-Depth Comparison of Models

Model Selection

Once the basics were mastered, I expanded my experiments by testing several models, including:

- **Llama2 and Llama3**: Open-source models developed by Meta, known for their flexibility and ability to handle complex prompts.

- **GPT-4**: A proprietary model by OpenAI, renowned for its power and completeness.
- **Alternative models**: Lighter models were evaluated but did not meet the requirements for accuracy and depth of responses.

Test Scenarios
To evaluate these models, I designed a series of semi-complex scenarios:

1. **Fact extraction**: For example, "What are the main causes of sepsis?"
2. **Synthesis and contextualization**: "Explain how sepsis affects the respiratory system."
3. **Justification of answers**: The models had to not only provide information but also explain their reasoning.

These scenarios allowed me to assess the models based on several criteria:

- **Answer accuracy**: The models had to provide accurate and relevant information.
- **Clear justifications**: A detailed explanation was essential to integrate the answers into a clinical context.
- **Response time**: I compared the response speed, considering the limitations of the APIs.

Phase 3: Results and Limitations

Results and Model Comparison
At the end of these tests, Llama emerged as the optimal choice for several reasons:

- **Accuracy and Justification**: Llama models provided clear answers, well-suited to the medical context, with well-structured justifications.
- **Adaptability**: The flexibility of prompts and the customization of parameters allowed for optimized performance.
- **Cost-efficiency**: Unlike GPT-4, Llama is open-source, reducing costs related to model use.

Limitations and Solutions
Some limitations were identified during these experiments:

1. **Generalized Trends**: Sometimes, the models returned generic answers, such as systematically identifying the central nervous system as an infectious focus. To address this, I refined the prompts and tested different API parameter configurations.
2. **Handling Long Clinical Notes**: Large notes posed problems by exceeding the token limits. I addressed this by segmenting the data or simplifying the clinical notes before processing.

Conclusion

This progressive exploration, from simple tests to complex scenarios, allowed me to select Llama as the primary model for the project. The knowledge gained during this phase was fundamental for efficiently integrating the model into the analysis of sepsis clinical cases, providing a solid foundation for future developments.

Mission in Hanover : Knowledge Graphs

Mission Objectives

As part of my internship, I had the opportunity to participate in a two-week training mission in Hanover, Germany, from late June to early July. This experience was organized thanks to a collaboration between my internship supervisor, Madame Ernestina Menasalvas, and a colleague researcher, Mrs. Maria-Esther VIDAL, who specializes in Knowledge Graphs in a German laboratory. The main objective of this mission was to introduce me to the theoretical and practical concepts of Knowledge Graphs and provide a broader view of the applications of artificial intelligence.

Knowledge Graphs (KG) are structures that allow data to be represented as nodes (entities) and edges (relations). Unlike a simple relational database, KGs directly link information to each other, creating a semantic network that facilitates exploration and querying. This model is particularly useful for organizing complex information, identifying implicit relationships between data, and answering advanced queries quickly and accurately.

This approach has become essential in fields like big data management, where interconnection is key. For example, in the medical research sector, KGs help link symptoms, diagnoses, treatments, and patient outcomes, providing an overview of clinical cases.

The specific objectives of the mission were:

1. **Acquire a solid theoretical foundation**: Understand the structure of Knowledge Graphs, their fundamental principles, and querying mechanisms using tools like SPARQL or Neo4j.
2. **Practice through guided exercises**: Learn how to build, visualize, and query simple graphs to master the technical tools required for manipulating them.
3. **Participate in specialized AI conferences**: Take advantage of events organized as part of this training to explore broader applications of artificial intelligence beyond Knowledge Graphs.

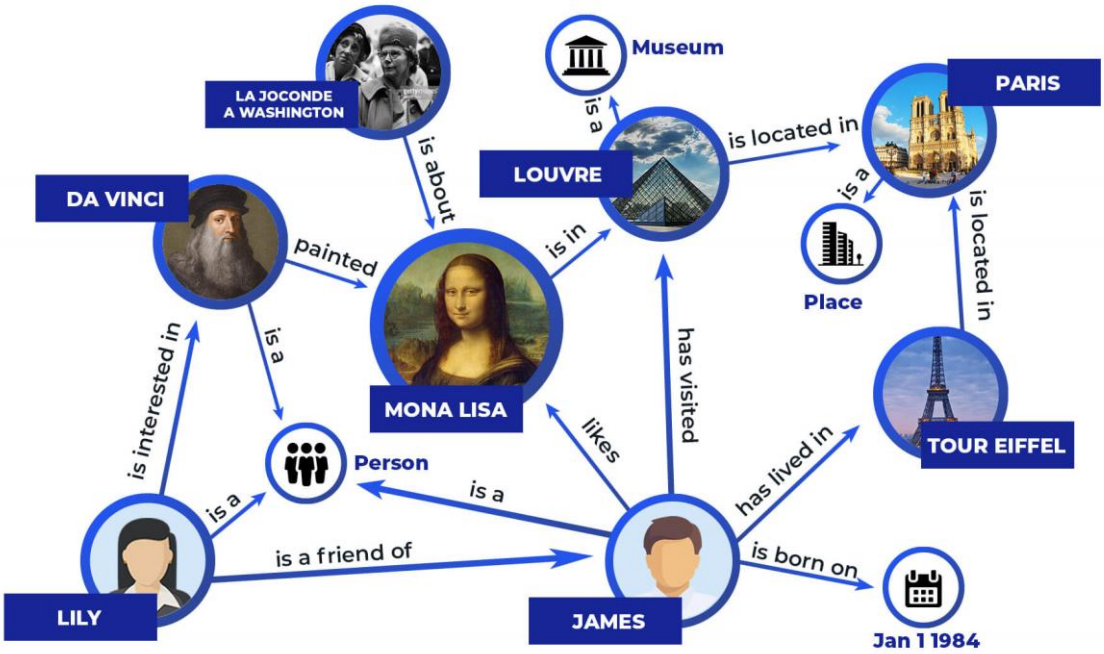

Image 3 : Example of a Knowledge Graph

Contributions to the Main Project

During this mission, I attended intensive courses led by specialized researchers. These sessions included:

- A detailed introduction to Knowledge Graphs : their theoretical foundations, use cases, and advantages over other data structuring methods.
- Practical workshops with progressive exercises focused on creating and querying graphs using generic datasets.
- Active participation in conferences on artificial intelligence, where I discovered current trends and innovations in the field, particularly the use of graphs in modeling complex systems.

Although these exercises were disconnected from the clinical cases I was studying in my main project, this training enriched my understanding of innovative approaches for handling complex data. For instance, Knowledge Graphs could be used in future analyses to structure relationships between information extracted from clinical notes, thus facilitating their use in decision support systems.

Moreover, the AI conferences helped me better understand the opportunities offered by this discipline in sectors like healthcare, education, and the creative industries. They also deepened my understanding of the interactions between different tools and analysis methodologies.

This experience had a significant impact on my technical skills, allowing me to quickly grasp complex concepts and collaborate in an international research environment. It also broadened my perspective on the possibilities of integrating advanced technologies like Knowledge Graphs into multidisciplinary research projects.

In conclusion, this mission was a formative and enriching step in my internship, both technically and professionally, and opened new perspectives for clinical data analysis.

Writing and Improving Prompts

Context and Objectives

The creation of precise and tailored prompts was essential to ensure that the API could reliably respond to critical clinical questions derived from the notes of patients with sepsis. A file provided by the doctors, containing all the information they wanted, served as the basis for identifying medical needs and determining which information to extract. This file contained detailed specifications from the doctors, such as the categories of information needed (e.g., types of cultures, pathogens, antibiotics) and the expected formats.

The objectives was twofold :

1. **Design prompts** that could extract relevant information while respecting the limitations of language models.
2. **Structure responses in a standardized manner** to facilitate subsequent analysis in Excel or JSON files.

Prompt Creation Process

1. **Iitial Analysis of Medical Needs**
   I started by exploring the Excel file to identify the priority categories of medical information. For example :
   - Types of cultures (blood, urine, respiratory, etc.)
   - Pathogens identified in blood culture or urine culture results.
   - Treatments administered, particularly antibiotics.
   - Specific indicators, such as the adequacy of treatments or the need for oxygen therapy.

This phase helped me understand the doctors' expectations regarding the responses the API was required to provide.

2. **Designing Initial Prompts** :
   Each category was then translated into a clear prompt structured around the following elements:
   - **Designing Initial Prompts** :
     Each category was then translated into a clear prompt structured around the following elements :
     - *"Indicate the type of positive culture in the clinical note. Select one of the following: 1, ORINA | 2, MUESTRA RESPIRATORIA | 3, etc."*
     - *"Identify the pathogens found in blood culture results. Respond with one of the following: 1, S. aureus | 2, E. coli | 3, etc."*
   - **Request for Systematic Justification :**
     Every response needed to be accompanied by a justification explaining how it was determined. For example :
     - *"The identified pathogen is: S. aureus. Justification: This was identified based on the microbiological results in the clinical note."*

- **Standardized Formats**: Numeric scales (e.g., 1-10 for pathogens) or binary answers ("Yes/No") were preferred to reduce ambiguity.

3. **Testing and Initial Adjustments** :
   The initial prompts were incorporated into the prompts.xlsx file and tested on real clinical notes. The tests revealed several issues :

   - Some formulations generated vague or off-topic answers.
   - Justifications were sometimes insufficient or irrelevant to the context.

For example, the initial prompt for blood culture results was too general :
"Indicate the pathogen found in the blood culture results."
After adjustment, it became :
"Identify the pathogen in the blood culture results. Select one of the following: 1, S. aureus | 2, SCN | 3, E. coli, etc. Justify your response based on the clinical note."

4. **Validation with Medical Teams** :
   I then shared the prompts with the medical teams for validation. Their feedback allowed me to fine-tune the formulations and introduce nuances specific to each category. For example::

   - For oxygen therapy, it was decided to include precise ratios of oxygen concentration.
   - For pathogens, the list was expanded to include less common but clinically significant microorganisms.

5. **Optimizing API Performance** :
   Each prompt was designed to maximize accuracy while respecting the token limits of the models. This involved :

   - Simplifying the prompts while maintaining clarity.
   - Testing the impact of linguistic variations on responses to select the most effective formulations.

Types of Prompts and Examples

- **Urine Culture Results**
  Prompt :
  "Indicate whether the urine culture results are positive or negative. Justify your response with specific findings from the note."
  Expected Response :
  "The urine culture results are: positive. Justification: Presence of E. coli colony in the microbiology section."

- **Oxygen Therapy**
  Prompt :
  “Was the patient on supplementary oxygen? Respond as 1, Sí or 2, Aire Ambiente. If Sí, provide the oxygen flow rate and concentration.”
  Expected Response :
  “1, Sí. Justification: The patient received oxygen at 4 L/min via nasal cannula.”

- **Blood Culture Pathogens**
  Prompt :
  "Identify the pathogen found in the blood culture results. Respond with one of the following: 1, S. aureus | 2, SCN | 3, etc."
  Expected Response :
  "2, SCN. Justification: Found in blood culture analysis from the note."

Results and Impact

The finalized prompts allowed :

1. **Structuring Extracted Results** : All information was organized into tables in Excel files, facilitating analysis.
2. **Improved Quality of Responses** : The added justifications enhanced the doctors' trust in the API-generated results.
3. **Optimized Clinical Workflow** : Standardized responses enabled direct integration into the clinicians' existing workflows.

Reflections and Future Directions

This phase of the project highlighted the importance of prompt engineering to tailor language models to specific use cases. Looking ahead, it would be interesting to :

- Automate prompt testing through simulations.
- Explore methods for dynamically generating prompts based on specific user needs.

In conclusion, the developed prompts were a cornerstone of the project, transforming clinical notes into actionable data while meeting the doctors' expectations.

id	description	prompt
1	Oxigenoterapia	You are reading a clinical note about a patient's oxygen therapy. The clinical note is written in Spanish and is accompanied by a radiography file that you can refer to if needed. Here are the contents of the note: {} You are one of the leading medical experts in respiratory care, and I need your help with an important medical task. Your task is to perform Named Entity Recognition (NER) to identify whether the patient received oxygen therapy, and if so, specify the type and flow rate. The response should follow one of the recognized oxygen therapy categories, which are: 1. Aire Ambiente 21% 2. GGNN 1 lpm 24% 3. GGNN 2 lpm 27% 4. GGNN 3 lpm 30% 5. GGNN 4 lpm 33% 6. GGNN 5 lpm 36% 7. VMask 5 lpm 40% 8. VMask 6 lpm 45% 9. VMask 7 lpm 50% 10. VMask 8 lpm 55% 11. Reservorio 8-9 lpm 60-65% 12. Reservorio 10 lpm 70% 13. Reservorio 11 lpm 75% 14. Reservorio 12 lpm 80% You must carefully review and evaluate the clinical note to identify the oxygen therapy type and flow rate. Pay attention to specific terms like "Aire Ambiente", "GGNN", "VMask", or "Reservorio", as well as any associated flow rates (e.g., 1 lpm, 2 lpm, etc.). Format your answer as follows: -"The patient received oxygen therapy: [yes/no]. -Type: [Select from the list above: Aire Ambiente, GGNN, VMask, Reservorio, etc.]. -Flow rate: [Select the appropriate lpm and corresponding oxygen percentage]. -Justification: [Explain how you identified the therapy type, referencing specific terms in the note and indicating the flow rate if mentioned]."

Image 4 : Example of how the prompts are structured

Development of the Clinical Notes Processing Application

Context

The development of the clinical analysis application is part of a project aimed at facilitating the detection and analysis of sepsis cases from clinical notes. The primary mission was to integrate language models (LLMs) to extract specific information from unstructured data.
The initial steps of the project involved testing various models, including Llama2 and Llama3, to select the one offering the best performance. The code architecture evolved over time through several iterations to improve the application's efficiency, scalability, and user-friendliness. The final version of the application features a modular structure and optimized functionalities, tailored to the end-users' needs.

Modules and Features

1. api.py: **API Call Management**
    - **Goal** : Provide an interface between the application and a language model, enabling prompt submission and retrieval of structured responses.
    - **Functionality** :
        - call_api :
            - Send a prompt along with parameters (maximum number of tokens, model used, etc.) to the API.
            - Retrieve the response in the form of a structured JSON message.
            - Handle network or JSON decoding errors.
        - extract_information :
            - Analyze the response to separate the main information from the justification.
            - If no justification is present, a default message is provided.
    - **Example of application** :
        - **Prompt** : " Identify the likely source of sepsis in the following text."
        - **API response** : "Focus : Foco biliar. Justification Based on an ultrasound showing biliary dilation."

```python
# Extrait de api_sepsis_project.py
response = requests.post(url, json={"prompt": prompt})
return response.json().get('message', {}).get('content', None)
```

Image 5 : Excerpt from the code api.py

2. file_reader.py **: Reading clinical files**

- **Goal** : Standardize the reading of clinical notes from files in various formats.
- **Operation** :
  - Detects the file type (TXT, JSON, CSV, Excel) based on its extension.
  - Converts the content into structured text, regardless of the original format.
- **Example of application** :
  - TXT file: Direct reading of the text.
  - Excel file: Converts relevant columns into a string of characters.
- **Importance** : This module ensures that all clinical notes are processed consistently, regardless of their input format.

```
# Extrait de file_reader_sepsis_project.py
if extension == '.txt':
    with open(file_path, 'r', encoding='utf-8') as file:
        return file.read()
```

Image 6 : Excerpt from the code file_reader.py

3. process.py **: Orchestration of the processing**

- **Goal** : Coordinate the different stages of clinical note analysis.
- **Operation** :
  - Reads files using file_reader.
  - Allocates tokens using token_manager.
  - Sends prompts to APIs for each clinical note.
  - Organizes the results (extracted information and justifications) by category.
- **Example of application** :
  - A list of 10 clinical notes is processed sequentially with different prompts. The results are grouped by category (e.g., Detected Pathogens, Provided Justifications).

```
# Extrait de process_sepsis_project.py
for file_path in file_paths:
    clinical_note = read_clinical_file(file_path)
    response_message = call_api(prompt)
    extracted_info, justification = extract_information(response_message)
```

Image 7 : Excerpt from the code process.py

4. token_manager.py **: Token Management**

    - **Goal** : Optimize the use of tokens to avoid errors related to the limits imposed by the API.
    - **Operation** :
        - calculate_token_count: Calculates the number of tokens in an input text.
        - allocate_tokens_for_api: Allocates the remaining tokens for the API responses.
    - **Example of application** :
        - A long clinical note containing 3000 tokens leaves 1096 tokens available for the API response, which is automatically calculated.

```python
# Extrait de token_manager_sepsis_project.py
def calculate_token_count(text):
    return len(tokenizer.encode(text))
def allocate_tokens_for_api(input_text, max_tokens):
    used_tokens = calculate_token_count(input_text)
    return max_tokens - used_tokens
```

Image 8 : Excerpt from the code token_manager.py

5. save_manager.py **: Gestion des sauvegardes**

    - **Goal** : Allow users to save results in suitable formats (Excel or JSON).
    - **Operation** :
        - save_to_excel :
            - Saves results in an Excel spreadsheet.
            - Creates distinct columns for the extracted information and their justifications.
        - save_to_json :
            - Saves results in a hierarchical JSON file.
            - Merges new results with existing data.
    - **Example of application** :
        - After analyzing 20 clinical notes, the user obtains an Excel file containing the columns: File Name, Clinical Note, Source, Justification (Source).

```python
# Extrait de save_results_json_sepsis_project.py
with open(output_file_path, 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=4)
```

```python
# Extrait de save_results_excel_sepsis_project.py
new_df = pd.DataFrame({'File Name': file_names, 'Clinical Note': clinical_notes
new_df.to_excel(output_file_path, index=False)
```

Image 9 : Excerpt from the code save_manager.py

6. ui.py **: User Interface**

- **Goal** : Provide a user-friendly interface for clinicians, allowing them to easily use the application without technical knowledge.
- **Operation** :
  - Select clinical files via a file explorer.
  - Choose prompts from a dedicated Excel file.
  - Select the backup format (Excel or JSON).
  - Display analysis steps and potential errors.
- **Example of application** :
  - A clinician selects 5 TXT files, applies 3 prompts, and saves the results in a JSON file named analyse_sepsis.json.

```python
# Extrait de ui_sepsis_project.py
class SepsisAnalysisApp(QWidget):
    def initUI(self):
        self.label = QLabel('Click the button to start processing clinical note
        self.startButton = QPushButton('Start')
        self.startButton.clicked.connect(self.start_processing)
        layout.addWidget(self.label)
        layout.addWidget(self.startButton)
```

Image 10 : Excerpt from the code ui.py

Practical Example

Sepsis Source Extraction :

- Clinical Note : " *80-year-old man with probable sepsis of biliary origin.*"
- Prompt: " *What is the source of sepsis?*"
- Result :
  - Foyer : "*SNC*".
  - Justification : "*Based on “neutroperia” and “SNC”*"

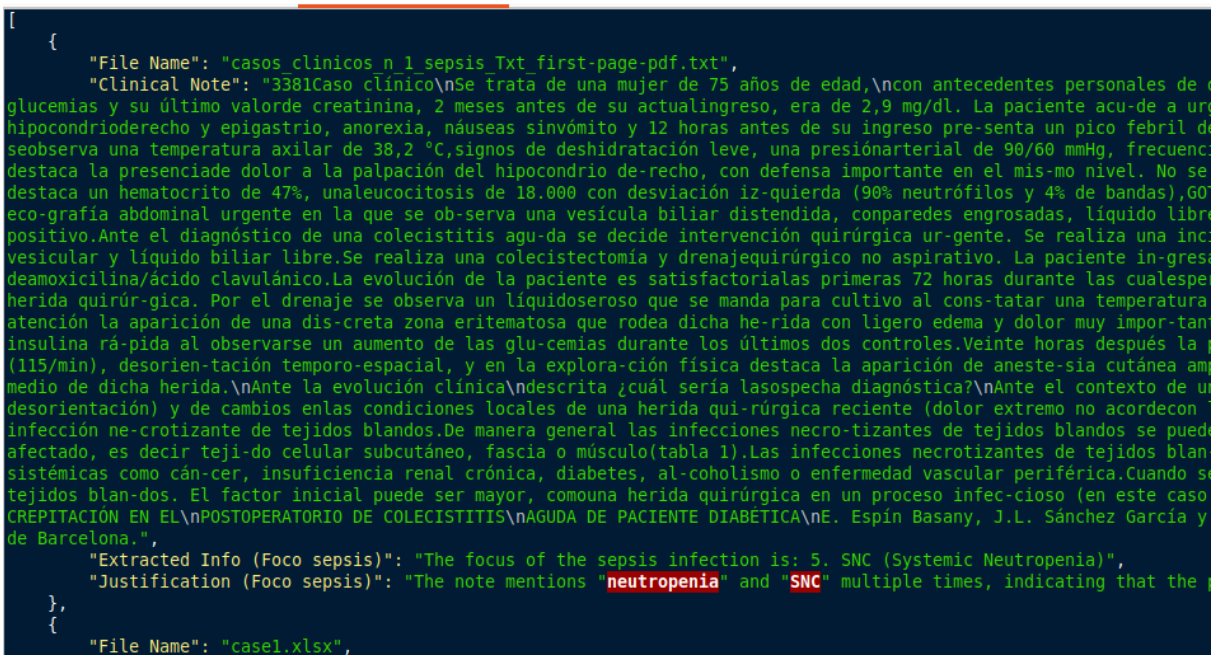

Image 11 : Excerpt from the JSON file containing the results of the clinical notes processing tests

Areas for Improvement

The application stands out for its ability to transform raw data into actionable information while ensuring maximum flexibility for users. It is a key tool for improving the management of sepsis cases in both clinical and research settings.

Here are some improvement points and ideas for further developing the application in the future :

1. Training Custom Models

- **Current :** The application relies on generic models (Llama2 or Llama3).
- **Proposed Improvement** Collect local clinical data to fine-tune an LLM model specific to the needs of partner hospitals.
- **Impact :** Increase the precision and relevance of analyses in a specific clinical context.

2. Performance Optimization

- **Current :** Token management and API calls are optimized but could be improved.

- **Proposed Improvement :**
    - Implement parallelization of API calls to speed up processing of batches of clinical notes.
    - Introduce adaptive prompt management to maximize extracted information while minimizing token usage.
- **Impact :** Reduce processing times for large data volumes.

3. Multilingual Support

- **Current :** The application is primarily adapted to clinical notes in Spanish.
- **Proposed Improvement :** Add automatic translation preprocessing via APIs like DeepL or Google Translate to analyze notes in multiple languages.
- **Impact :** Make the tool accessible to an international community of clinicians.

4. Workflow Automation

- **Current :** The user must manually select files, prompts, and save formats.
- **Proposed Improvement :** Introduce a fully automated mode where files added to a monitored folder are automatically processed and saved in a predefined format.
- **Impact :** Increase efficiency in hospital environments where time is critical.

5. Data Visualization

- **Current :** Results are presented in tabular form (Excel, JSON).
- **Proposed Improvement :**
    - Integrate visualization tools like interactive graphs or dashboards (via Dash or Streamlit).
    - Add heatmaps to visualize sepsis sources across populations.
- **Impact :** Help clinicians quickly interpret data for decision-making.

6. Data Security and Privacy (important for medical confidentiality)

- **Current :** There is no specific data security management in place.
- **Proposed Improvement :**
    - Add encryption for sensitive files and communications with the API.
    - Implement access control mechanisms to restrict the use of the application to authorized users.
- **Impact :** Strengthen compliance with medical data protection regulations (GDPR, HIPAA).

7. Cloud Deployment and REST API

- **Current :** The application appears designed for local use.
- **Proposed Improvement :**
    - Host the application on a cloud server to allow remote access.
    - Provide a REST API that exposes key functionalities (analysis, backup, etc.) for integration into hospital management systems (HIS).
- **Impact :** Increase scalability and ease of use.

8. User Interface Enhancement

- **Current :** The user interface is functional but basic.
- **Proposed Improvement :**
    - Add customization options for users (color schemes, themes).
    - Allow for the creation and saving of predefined workflows (e.g., applying a specific set of prompts).
- **Impact :** Enhance usability and adoption of the tool.

These improvements could not only strengthen the efficiency and versatility of the application but also provide a strategic vision for its large-scale deployment in diverse clinical environments.

Token Analysis

Context

Token analysis was a crucial aspect of my internship, aiming to optimize the performance of the API used to analyze clinical notes regarding sepsis. Each natural language model (LLM) used for these analyses has specific constraints on the number of tokens it can process in a single request. Therefore, it was imperative to compare and select the most suitable tokenization method to ensure efficient and accurate data processing.

Methodology

1. Comparison of Tokenization Methods

To meet this need, I explored several tokenization methods :

- AutoTokenizer (Hugging Face) : A generic tokenizer that adapts to various models.
- LlamaTokenizer and LlamaTokenizerFast : Specific to Llama models, these tokenizers offer optimized performance for Meta's models.
- Tiktoken : Mainly used with OpenAI models for its high performance.
- Tokenisation Manuelle : Implemented using regex to split texts into basic tokens.

Each method was tested on the same clinical notes to evaluate :

- The number of tokens generated.
- Compatibility with language models.
- Overall performance.

2. Development of Comparison Scripts

I developed several Python scripts, each dedicated to a tokenization method. The scripts included :

- Reading and preprocessing clinical notes in various formats (TXT, JSON, Excel).
- Token count calculation by method.
- A user interface (PyQt5) to select files and view results.

3. Result Analysis

The results were saved in an Excel file for comparative analysis. A chart was also generated to visualize the differences between the methods. This process allowed me to identify notes that exceeded the allowed token limit, which is critical for optimizing interaction with the API.

Results

The analyses showed that :

- **Tiktoken** was the most efficient in terms of speed and compatibility with OpenAI models.
- **LlamaTokenizerFast** provided similar performance with better integration for Llama models.
- AutoTokenizer produisait plus de tokens, ce qui pourrait entraîner des dépassements de limite dans certaines situations.
- **Manual Tokenization**, while educational, lacked robustness and precision.

Considering the API's constraints and the performance of the different methods, **LlamaTokenizerFast** was selected as the primary method for the project going forward, as it integrated perfectly with the Llama models used (especially Llama-3).

This phase allowed me to better understand the importance of tokenization in clinical note processing. The results obtained served as the foundation for optimizing API calls and ensuring that each clinical note was analyzed efficiently without exceeding the token limit. In the future, integrating text compression or summarization techniques could further enhance these performances.

Example of how the comparison between different tokenizers was done :

File Name	Number of characters (with space)	Tokenizer used	Number of tokens	Explanation	Estimated token count
casos_clinicos_n_1_sepsis_Txt_first-page	4732	Tiktoken	1781	Works only for OpenAI models (with gpt-2)	938
casos_clinicos_n_1_sepsis_Txt_first-page	4732	Tiktoken	1470	Same With cl100k_base	
casos_clinicos_n_1_sepsis_Txt_first-page	4732	Manual	938		
casos_clinicos_n_1_sepsis_Txt_first-page	4732	LlamaTokenizer	1695	With Llama-2-7b-hf (because it doesn't work with Llama-3)	
casos_clinicos_n_1_sepsis_Txt_first-page	4732	AutoTokenizer	1468	With Llama-3.2-1B	
casos_clinicos_n_1_sepsis_Txt_first-page	4732	LlamaTokenizerFast	1468	With Llama-3.2-1B	
casos_clinicos_n_1_sepsis_Txt_first-page	4732	AutoTokenizer	1468	With Llama-3.2-2B	
casos_clinicos_n_1_sepsis_Txt_first-page	4732	LlamaTokenizerFast	1468	With Llama-3.2-2B	
casos_clinicos_n_1_sepsis_Txt_first-page	4732	AutoTokenizer	1468	With Llama-3.2-3B	

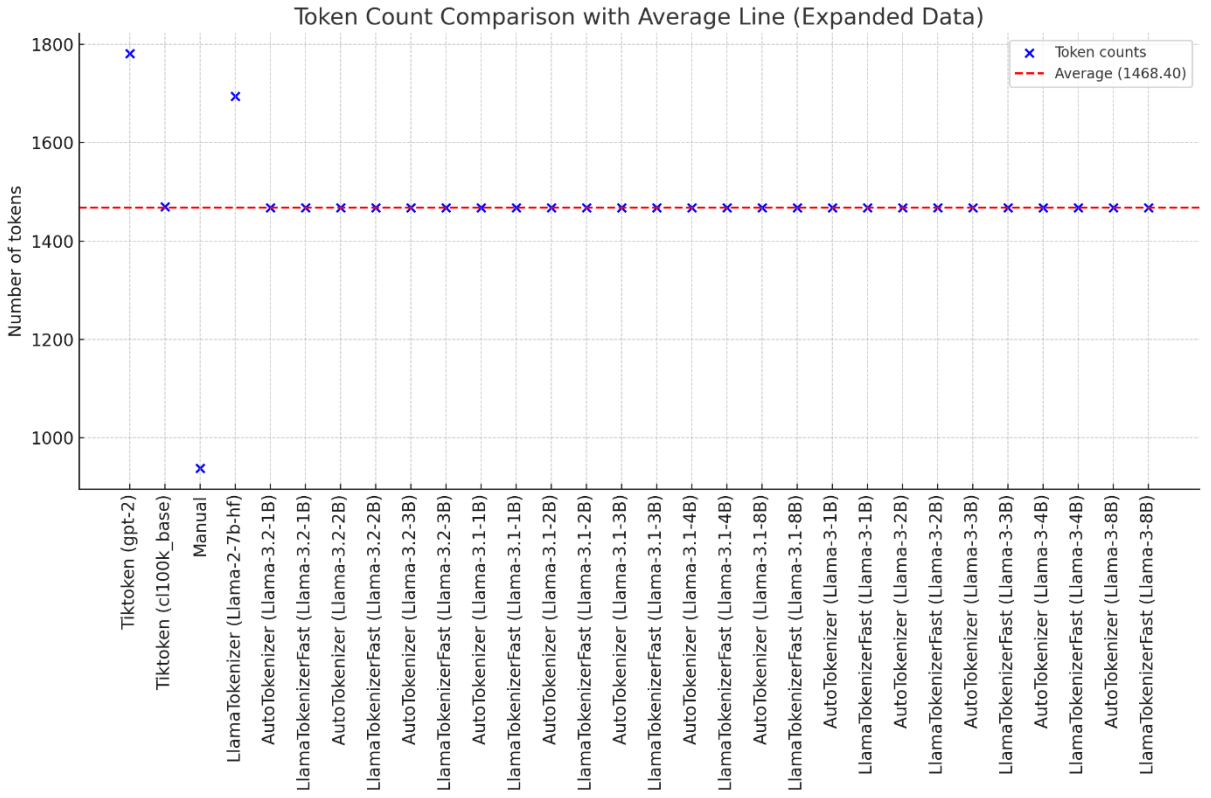

Image 12 : Comparative Table and Graph of the Different Tokenizers

Internship Closure

The final phase of my internship at the MEDAL laboratory at UPM in Madrid focused on finalizing the features developed throughout the mission and setting up unit tests to ensure the robustness and reliability of the clinical analysis application. Below are the main steps completed during this phase:

Finalization of Features

1. **Consolidation of Core Modules**:
    - The various modules for processing clinical notes, token management, and result saving were seamlessly integrated to create a robust and modular application.
    - Particular attention was given to optimizing the performance of the API by dynamically adjusting the parameters of API calls based on token limits.
2. **Standardization of Output Formats**:
    - The application now allows saving results in two main formats: Excel and JSON. Each format was tested to ensure data integrity during writing, notably by avoiding duplicates when adding new analyses to existing files.
3. **User Interface (UI) Improvement**:
    - An intuitive user interface, based on PyQt5, was finalized to allow simplified navigation between different steps: file selection, prompt choice, data processing, and result saving.
4. **Prompt Validation**:
    - Specific clinical analysis prompts were refined, particularly for critical questions such as identifying the type of culture or the administered antibiotic treatment. Each prompt was tested with different cases to validate its accuracy and relevance.

Unit Test Setup

To ensure the stability and scalability of the project, a series of unit tests was developed and executed on the various modules of the application:

1. **API Module Tests**:
    - Verification of correct API responses, network error handling, and cases of missing justification in the messages.
2. **File Reading Module Tests**:
    - Validation of compatibility with different clinical file formats: .txt, .json, .xlsx, and .csv.
    - Error handling for unsupported or corrupted files.
3. **Token Management Module Tests**:
    - Accurate token count calculation for each clinical note.
    - Dynamic allocation of remaining tokens for API responses while respecting imposed limits.
4. **Save Modules Tests**:
    - Verification of writing processes in Excel and JSON files.

- Handling data merging in existing files while avoiding duplicates.

Validation and Feedback

After implementing all features and testing their proper functioning, a validation phase was carried out in collaboration with the MEDAL laboratory team. This phase allowed:

- Receiving feedback on the application's ergonomics and the relevance of the results.
- Identifying potential future improvements, such as adding new categories of analysis or integrating with existing clinical databases.

Retrospective

This final part marks the conclusion of a rewarding project, both technically and scientifically. The results laid the groundwork for the future use of the application in real clinical environments. Additionally, the positive feedback from the research team paves the way for potential collaborations to further improve and extend the project.

The closing phase of my internship was also marked by several technical and personal challenges, which, although demanding, enriched my experience and enhanced my skills.

One of the main technical difficulties lay in exploring language models (LLMs). Learning advanced LLM concepts, such as token management, effective prompt construction, and interpreting model responses, required considerable adaptation time. However, this allowed me to deeply understand how models like Llama work and their applications in a clinical context.

Starting this internship in a new country, Spain, presented an additional challenge, particularly with the language barrier. Although I didn’t master Spanish, this experience led me to rely on English as the primary language of communication with my colleagues and supervisors. This immersion significantly improved my English skills, both professionally and personally, while broadening my cultural perspective.

A major challenge arose at the end of July when the laboratory’s servers, essential for running API queries, had to be replaced. This temporarily halted the development of the Sepsis application, as my personal computer lacked the necessary power to perform these complex calculations.

To address this constraint, I decided to pause the Sepsis application and redirect my efforts toward:

- Improving the prompts to make them more accurate and effective.
- Studying other AI APIs and models in depth to compare their performance and determine the best options for the project.

When the servers were ultimately not brought back online, I had to adjust my approach again. I continued development and testing without actual API query results by simulating the expected responses. Meanwhile, a colleague with a more powerful computer was able to execute some queries to validate the functionality of critical modules.

These challenges taught me resilience and adaptability in the face of unforeseen situations. I learned how to prioritize essential tasks to move the project forward despite constraints and collaborate effectively with my colleagues to overcome these obstacles. This experience also made me more aware of the importance of anticipating unforeseen issues in project management and always considering alternative plans.

General Conclusion

My final-year internship at the MEDAL laboratory, located at the Biomedical Technology Center in Madrid, was a pivotal period in both my academic and professional journey. This internship not only consolidated my theoretical knowledge but also allowed me to apply it in a practical and impactful context, with a focus on the analysis of clinical notes related to sepsis.

During this internship, I developed an application dedicated to the analysis of complex clinical data, integrating advanced language models like Llama3. This application significantly improves the diagnostic process by autonomously and accurately extracting essential information from clinical notes. The effectiveness of this solution was tested and validated, demonstrating its ability to reduce human errors and accelerate clinical decision-making. The implementation of technologies such as Python, PyQt5, and various AI libraries enhanced my technical profile and my ability to solve complex problems.

This internship was also a great opportunity to develop my interpersonal skills. Working within a multidisciplinary team taught me the importance of communication, knowledge sharing, and close collaboration with experts from various fields, from computer science to medicine. My integration into the team and my active participation in meetings and project decisions were key to the success of the tasks entrusted to me.

A significant part of my learning was reflecting on the ethics of using AI in medicine. The need to reconcile technological innovation with respect for human dignity guided the development of the application, establishing strict protocols for protecting patient data and ensuring complete transparency in AI-assisted decision-making processes.

This internship confirmed my interest in the digital health sector and opened career prospects in innovative fields related to artificial intelligence. I am now more determined to pursue this path, further exploring how disruptive technologies can transform healthcare. The end of my internship marks the beginning of a deep commitment to using my skills to improve healthcare systems through the careful integration of advanced technologies.

In summary, this internship was a transformative experience that expanded my understanding of the practical applications of computer science in a medical context, strengthened my technical and interpersonal skills, and sharpened my awareness of the ethical implications of my work. I am grateful for the learning opportunities provided by the MEDAL laboratory and optimistic about the future of technology in medicine.

Bibliography

Projects and Developed Modules

1. **Clinical Note Analysis for Sepsis**
    - **Description** : Development of a clinical note analysis system using language models (LLM) to extract critical information about sepsis.
    - **Key Features** :
        - Graphical user interface based on PyQt5.
        - API integration with dynamic token management.
        - Saving results in Excel and JSON formats.
    - **References** :
        - Main file : *main_sepsis_project.py*.
        - Modules associés : *process_sepsis_project.py, save_results_excel.py, file_reader.py*.
2. **Tokenization Benchmark for Clinical Notes**
    - **Description** : Comparaison de méthodes de tokenisation pour optimiser la compatibilité avec des modèles comme Llama3.
    - **Key Features** :
        - Tokenizers benchmark : AutoTokenizer, LlamaTokenizerFast, Tiktoken, méthode manuelle.
        - Performance analysis through dedicated scripts, such as *autotokenizer_cal.py* and *llamatokenizer_cal.py*.
    - **References** :
        - Results documented in : tokenizer_stats_tokenize_compare_project.xlsx.

Technologies Used

- **Languages and frameworks** :
    - Python : Development of scripts for clinical analysis and token management.
    - PyQt5 : Interactive graphical interfaces for analysis and visualization of results.
- **Libraries and APIs** :
    - Transformers (Hugging Face): Tokenizer management and integration with models like Llama3.
    - Tiktoken: Specialized tool for tokenizing clinical notes.

Relevant Training and Certifications

1. **Coursera - Deep Learning Specialization**
    - **Description** : Andrew Ng's course covering the basics of deep learning.
    - **Link** : Deep Learning Specialization.
2. **Stanford University - CS224N: Natural Language Processing with Deep Learning**
    - **Description** : Advanced course on natural language processing and LLM models.
    - **Link** : CS224N.
3. **Hugging Face - Transformers & NLP**
    - **Description** : Practical training on the use of pre-trained models and Transformers.
    - **Link** : Hugging Face Course.
4. **Fast.ai - Practical Deep Learning for Coders**
    - **Description** : A course for developers on practical applications of deep learning.
    - **Link** : Fast.ai Course.
5. **Google - Machine Learning Crash Course**
    - **Description** : Introduction to key concepts in machine learning.
    - **Link** : ML Crash Course.

Books and Additional Resources

1. **Books** :
    - *Deep Learning* by Ian Goodfellow, Yoshua Bengio and Aaron Courville.
    - *Natural Language Processing with PyTorch* by Delip Rao and Brian McMahan.
2. **GitHub ressources**:
    - Awesome AI
    - Awesome NLP
3. **Blogs and forums** :
    - Towards Data Science
    - Hugging Face Forum

Contributions and Supervision

- **Supervisors** :
    - Pr. Ernestina Menasalvas, **MEDAL Laboratory**, Madrid.
    - Alberto Gonzalez Calatayud, **MEDAL Laboratory**, Madrid.
    - Enrique SOLERA NAVARRO, **MEDAL Laboratory**, Madrid.
- **Lead Developer** : Thomas Fuzellier.